AI IDE implementation guide

03/31/2026 12:13:04

Give me an implementation guide for AI ide for this project

Onelab AI Fitness Assessment — Implementation Guide Understanding the Proble…

Create an implementation md file

View: Check available skills Create File: Creating implementation guide markdown f…

Create the whole project in this chat full omplemetation ask necessary questions

Let me ask a few quick questions before I build everything out. Tool: Generating…

Q: What should the UI look like? A: Dashboard with charts and summary cards Q:…

I'll build the full project now. Let me start by reading the frontend design skill. Vie…

Continue

Bash Tool: Install dependencies Bash Tool: Run all tests All 15 tests pass. Now verif…

explain the project the problem the suggested solution that wea are building it…

ReconcileIQ — Explained for Beginners The Real-World Problem Imagine you run a…

How to sumbit the project according to the photo

Looking at the photo again carefully: What to Submit (5 Deliverables) 1. 📝…

Onelab — AI Fitness Assessment

Time: 2 hours | **Tools:** Any AI | **Right answer:** There isn't one

The Problem

A payments company's books don't balance at month end. They know every transaction their platform processed. They know what the bank says actually arrived. The two should match. They don't. Find out why. Show them where the gaps are.

Test Data

No files provided. Generate your own — that's part of the assessment. A payments platform records a **transaction** instantly when a customer pays. The bank batches and **settles** funds 1–2 days later. At month end every transaction should have a matching settlement.

Plant these gap types: a transaction that settled the following month · a rounding difference that only shows when summed · a duplicate entry in one dataset · a refund with no matching original transaction.

State your assumptions. There are no wrong ones as long as you name them.

What to Submit

1 **Brainstorming thread** Full LLM convo. Do not clean it up.

2 **Distilled prompt** The single prompt you fed into Claude Code or Cursor or any other AI coding tool.

3 **Claude Code thread** Full execution thread including wrong turns.

4 **Test cases** What you wrote to verify your solution works.

5 **Working output** Deployed link + code zip + Demo video + 3 sentences on what it would get wrong in production.

Give me an implementation guide for AI ide for this project

3/31/2026, 11:21:11 AM

Onelab AI Fitness Assessment — Implementation Guide

Understanding the Problem

You're building a **payments reconciliation tool** that compares two datasets:

- **Transactions** — what the platform recorded when customers paid
- **Settlements** — what the bank actually deposited (batched 1–2 days later)

The tool finds 4 planted gap types: late settlements, rounding differences, duplicates, and orphan refunds.

Assumptions (state these in your submission)

- Transactions are in local currency, settlements in the same currency
- Settlement batches can contain multiple transactions
- "Month end" means calendar month boundary
- Rounding differences are ≤ $0.02 per transaction
- Refunds are negative-amount transactions

Step 1 — Generate Test Data

Prompt for your AI IDE:

```
Generate two CSV files for a payments reconciliation exercise.
```

```
transactions.csv columns:
  transaction_id, customer_id, amount, currency, created_at,
status, type

settlements.csv columns:
  settlement_id, transaction_id, settled_amount, settled_at,
bank_reference

Plant exactly these anomalies:
1. One transaction from 2024-01-30 that settles on 2024-02-02
(next month)
2. One transaction where settled_amount differs by $0.01
(rounding)
3. One duplicate transaction_id in transactions.csv with
identical fields
4. One refund (negative amount, type="refund") with no matching
original transaction_id

Generate 50 clean rows plus the 4 anomalies. Output valid CSV
only.
```

Step 2 — Core Reconciliation Logic

Distilled prompt for Claude Code / Cursor:

```
Build a Python reconciliation engine:

Inputs: transactions.csv, settlements.csv
Logic:
  - Match transactions to settlements on transaction_id
  - Flag: settlement.settled_at month != transaction.created_at
month
```

```
  - Flag: abs(settlement.settled_amount - transaction.amount) >
0.005
  - Flag: duplicate transaction_ids (same id, any fields)
  - Flag: refunds (type=="refund") where original transaction_id
not found
  - Unmatched transactions (no settlement at all)

Output: gaps_report.json with structure:
  { gap_type, transaction_id, details, amount_diff }

Also output: summary with counts per gap type and total $
exposure
```

Step 3 — App Structure

```
reconciler/
├── data/
│   ├── transactions.csv
│   └── settlements.csv
├── engine/
│   ├── loader.py       # CSV ingestion + validation
│   ├── matcher.py      # join logic
│   └── detectors.py    # one function per gap type
├── api/
│   └── main.py         # FastAPI: POST /reconcile, GET /report
├── ui/
│   └── index.html      # drag-drop upload + results table
├── tests/
│   └── test_detectors.py
└── README.md
```

Step 4 — Test Cases to Write

Tell your AI IDE to generate these 4 unit tests:

Test	Input	Expected
`test_late_settlement`	txn Jan 30, settled Feb 2	flagged as late
`test_rounding_gap`	txn $100.00, settled $99.99	flagged, diff=$0.01
`test_duplicate_transaction`	same txn_id twice	1 duplicate flagged
`test_orphan_refund`	refund with unknown txn_id	flagged as orphan

Step 5 — What It Gets Wrong in Production (3 sentences)

Pre-write this for your submission:

> *The matcher assumes transaction_id is a reliable join key, but real payment processors sometimes remap IDs during retries, causing false-positive "unmatched" flags. Rounding detection uses a fixed $0.01 threshold, which breaks for non-USD currencies with different decimal conventions. The duplicate detector compares full-row equality, so a legitimate re-submission with a corrected amount would be silently missed.*

Submission Checklist

- ☐ Raw brainstorming chat (this conversation counts — don't clean it)
- ☐ Distilled prompt from Step 2 above
- ☐ Claude Code execution thread with wrong turns kept in

- [ ] 4 test cases from Step 4
- [ ] Deployed link + code zip + demo video + 3 sentences (Step 5)

3/31/2026, 11:21:38 AM

Create an implementation md file

3/31/2026, 11:21:51 AM

```
View: Check available skills
```

```
Create File: Creating implementation guide markdown file
```

```
Present File: /mnt/user-data/outputs/IMPLEMENTATION.md
```

Here's your implementation guide. It covers all 6 steps end-to-end: generating test data with the 4 planted anomalies, the full Python engine with one detector per gap type, the FastAPI layer, test cases, the distilled prompt to feed into your AI IDE, and the 3 production-weakness sentences ready to paste into your submission.

> *File: /mnt/user-data/outputs/IMPLEMENTATION.md*

3/31/2026, 11:22:56 AM

Create the whole project in this chat full omplemetation ask necessary questions

3/31/2026, 11:32:34 AM

Let me ask a few quick questions before I build everything out.

```
Tool: Generating ask_user_input_v0...
```


```
Tool: Generating ask_user_input_v0...
```


Here are a few quick questions before I build the full project:

1. What should the UI look like?

- Simple drag-drop upload + results table
- Dashboard with charts and summary cards
- CLI only (no UI needed)

2. Deployment target?

- Local only (just run it)
- Deploy to Render/Railway (free)
- Docker container

3. What should the gap report output be? *(pick all that apply)*

- JSON file
- CSV export
- Visual table in browser
- Email summary

3/31/2026, 11:32:49 AM

Q: What should the UI look like? A: Dashboard with charts and summary cards

Q: Deployment target? A: Deploy to Render/Railway (free)

Q: What should the gap report output be? (Select all that apply) A: CSV export, JSON file, Visual table in browser, Email summary

3/31/2026, 11:33:17 AM

I'll build the full project now. Let me start by reading the frontend design skill.

```
View: Reading frontend design skill
```

Now let me build everything.

```
Bash Tool: Setup project structure
```

```
Create File: Generate test data script with all 4 anomalies planted
```

```
Bash Tool: Clean and recreate project structure
```

```
Create File: Generate test data with 4 planted anomalies
```

```
Bash Tool: Generate test CSV data
```

```
Create File: Engine loader module
```

```
Create File: Detectors — one function per gap type
```

```
Create File: Matcher — orchestrates all detectors and builds
```

```
final report
```

```
Bash Tool: Create engine package init
```

```
Create File: FastAPI main application with all endpoints
```

```
Bash Tool: Create api package init
```

Now let me build the full dashboard UI:

```
Create File: Full dashboard UI - dark industrial finance
aesthetic with charts, summary cards, gap table
```

```
Bash Tool: Copy sample data to static folder for "Load Sample
Data" button
```

```
Create File: Full pytest test suite — 4 gap detectors +
integration
```

```
Bash Tool: Create tests package init
```

```
Create File: requirements.txt
```

```
Create File: Render deployment config
```

```
Create File: Procfile for Railway deployment
```

```
Create File: README with full setup, run, deploy instructions
```

Let me install dependencies and run the tests:

Continue

3/31/2026, 11:39:10 AM

```
Bash Tool: Install dependencies
```

```
Bash Tool: Run all tests
```

All 15 tests pass. Now verify the API works end-to-end:

```
Bash Tool: Smoke-test the live API
```

```
Bash Tool: Start server in background
```

3/31/2026, 11:46:59 AM

explain the project the problem the suggested solution that wea are building it functions from a beginners perspective

3/31/2026, 11:47:32 AM

ReconcileIQ — Explained for Beginners

The Real-World Problem

Imagine you run an online store. Every time a customer pays, your system writes it down instantly:

> *"Customer paid $100 at 2pm on Jan 15"*

But the **bank doesn't give you that money immediately**. The bank collects many payments throughout the day, bundles them together, and sends you one lump sum 1–2 days later. This is called a **settlement**.

At the end of the month, your finance team sits down and asks:

> *"Every payment our platform recorded — did the bank actually send us that money?"*

They compare two lists:

Your Platform Says	Bank Says
55 transactions totalling $24,830	53 settlements totalling $24,791

They don't match. Why not? Where's the $39?

That's the problem. Finding the answer manually — row by row across thousands of transactions — takes days. We're building a tool that does it in seconds.

The 4 Types of Gaps We're Looking For

Think of these like 4 categories of mistakes:

1. 🕐 Late Settlement

What happened: A customer paid on Jan 30, but the bank didn't send the money until Feb 2.

Why it's a problem: Your January books show the income, but the money arrived in February. An accountant sees a gap and panics.

Real analogy: You sold something before month-end, but the cheque cleared after month-end.

---

2. 💰 Rounding Gap

What happened: Customer paid $100.00, but the bank only sent $99.99.

Why it's a problem: One cent sounds harmless, but if this happens across 10,000 transactions, you're missing $100. Banks and payment processors sometimes round differently.

Real analogy: A cashier gives you $9.99 change instead of $10.00. Tiny, but it adds up.

---

3. 👯 Duplicate Transaction

What happened: The same transaction appears **twice** in your records with the same ID.

Why it's a problem: Your books show you received $200 from one customer, but you actually only received $100. Or worse — you accidentally charged them twice.

Real analogy: A receipt printer jams and prints the same receipt twice. Now your records are wrong.

---

4. 👻 Orphan Refund

What happened: Your system recorded a refund (giving money back to a customer), but there's no original charge to match it to.

Why it's a problem: Someone may have issued a fraudulent refund, or a system error created a refund for a transaction that never existed.

Real analogy: A shop issues a refund receipt, but there's no original purchase receipt anywhere in the system.

What We're Building — The Solution

We're building **ReconcileIQ** — a web app with 3 layers:

```
┌──────────────────────────────────────────┐
│            BROWSER (Dashboard)           │  ← What the user sees
│   Charts · Cards · Gap Table · Export    │
└──────────────────┬───────────────────────┘
                   │ uploads 2 CSV files
┌──────────────────▼───────────────────────┐
│             API (FastAPI server)         │  ← The middleman
│    Receives files, returns results       │
└──────────────────┬───────────────────────┘
                   │ runs analysis
┌──────────────────▼───────────────────────┐
│            ENGINE (Python logic)         │  ← The brain
```

```
│   loader · detectors · matcher          │
└──────────────────────────────────────────┘
```

Every File Explained

📁 `data/generate.py`

What it does: Creates fake but realistic test data — 50 normal transactions with the 4 problem types deliberately planted inside.

Why we need it: The assessment says "no files provided, generate your own." This script creates both CSV files we need to test with.

📁 `engine/loader.py`

What it does: Reads the two CSV files and checks they have the right columns.

Beginner analogy: Like a bouncer at a door — it checks your ID (the column names) before letting data into the system. If the file is missing a column like `transaction_id`, it throws an error immediately rather than crashing later in a confusing way.

📁 `engine/detectors.py`

What it does: Contains 5 functions — one per gap type. Each function takes the two datasets, looks for its specific problem, and returns a list of rows that are suspicious.

Beginner analogy: Like 5 specialist doctors. You don't ask your cardiologist to check your teeth. Each detector only looks for its one thing:

```
detect_late_settlements()     → checks dates
detect_rounding_gaps()        → checks amounts
detect_duplicates()           → checks for repeated IDs
detect_orphan_refunds()       → checks refunds have matching
charges
detect_unmatched()            → checks every transaction has a
settlement
```

📁 `engine/matcher.py`

What it does: Calls all 5 detectors, combines their results into one list, and calculates summary numbers (total gaps, total money at risk).

Beginner analogy: The team manager who collects reports from all 5 specialists and writes the single summary report for the boss.

📁 `api/main.py`

What it does: Creates a web server with URL endpoints your browser can talk to.

URL	What it does
`POST /reconcile`	Accepts the 2 CSV files, runs the engine, returns results
`GET /report/json`	Downloads the results as a JSON file
`GET /report/csv`	Downloads the results as a spreadsheet
`POST /report/email`	Emails the results to someone
`GET /health`	Just confirms the server is running

Beginner analogy: Like a restaurant kitchen. The browser is the customer, the API is the waiter, the engine is the chef. The waiter takes your order (files), gives it to the chef (engine), and brings back the food (results).

📁 `static/index.html`

What it does: The entire visual dashboard — built in one HTML file with CSS and JavaScript.

What the user sees:

- **Upload area** — drag and drop your two CSV files
- **4 stat cards** — total transactions, settlements, gaps found, money at risk
- **2 charts** — a donut chart (gaps by type) and a bar chart (money at risk by type)
- **Filter pills** — click to show only one type of gap
- **Gap table** — every single gap, searchable and sortable
- **Export buttons** — download as JSON, CSV, or email it

📁 `tests/test_detectors.py`

What it does: 15 automated tests that verify every detector works correctly.

Why tests matter: Imagine you fix the rounding detector and accidentally break the duplicate detector. Without tests, you'd never know until a real client's books are wrong. With tests, you run one command and instantly see what broke.

Beginner analogy: Like a checklist a pilot runs before takeoff. Every test is one item on the checklist.

📁 `render.yaml` + `Procfile`

What they do: Configuration files that tell Render and Railway (free cloud hosting platforms) how to start our app.

Beginner analogy: Like an instruction manual that comes with a piece of furniture. The hosting platform reads it and knows exactly what to do without you having to explain anything.

The Full Flow, Step by Step

```
User uploads transactions.csv + settlements.csv
            │
            ▼
API receives the files → passes to engine
            │
            ▼
loader.py validates columns → rejects bad files early
            │
            ▼
detectors.py runs all 5 checks simultaneously
            │
      ┌─────┴─────┐
  Found gaps    No gaps
      │
      ▼
matcher.py combines results + calculates totals
            │
            ▼
API returns JSON to browser
            │
            ▼
Dashboard renders:
```

```
  • Stat cards update
  • Charts draw
  • Gap table populates
  • Export buttons activate
```

What "15 Tests Passing" Means

When you run `pytest tests/ -v` and see this:

```
✓ test_flags_cross_month_settlement
✓ test_ignores_same_month_settlement
✓ test_flags_one_cent_diff
✓ test_does_not_flag_exact_match
✓ test_flags_duplicate_transaction_id
✓ test_flags_refund_with_no_original
... (15 total)
```

It means every single scenario we designed the tool to handle — it handles correctly. Green across the board.

The 3 Weaknesses (Honest Assessment)

Every system has blind spots. Ours has 3:

1. **ID remapping** — If a payment processor retries a failed payment and gives it a *new* ID, we'll flag the original as "unmatched" even though it was actually settled under the new ID.

2. **Currency assumptions** — Our $0.005 rounding threshold only makes sense for USD. Japanese Yen has no decimal places, so even a ¥1 difference would look like a massive

rounding gap.

3. **Smart duplicates** — If someone submits the same transaction twice but changes the amount (e.g. correcting a mistake), our duplicate detector misses it because the IDs differ. We'd count it as two legitimate transactions.

These aren't bugs — they're known limitations that would require more business logic to solve in a production system.

3/31/2026, 11:48:24 AM

How to sumbit the project according to the photo

3/31/2026, 11:56:55 AM

Looking at the photo again carefully:

What to Submit (5 Deliverables)

---

1. 📝 Brainstorming Thread

Full LLM conversation. Do not clean it up.

> ***This entire chat is your submission for this.*** *Screenshot or export this full Claude conversation — wrong turns, questions, explanations and all. Don't edit anything out. They want to see how you think with AI, not a polished version.*

How to export: On Claude.ai → click the conversation title → there's no direct export, so either:

- Screenshot every message (phone or desktop)
- Select all text → copy → paste into a Google Doc
- Use browser Print → Save as PDF

---

2. 🎯 Distilled Prompt

The single prompt you fed into Claude Code / Cursor / any AI coding tool.

Use this exact prompt — it's already in your `IMPLEMENTATION.md`:

```
Build a Python payments reconciliation tool.
```

```
Inputs: transactions.csv (transaction_id, customer_id, amount,
currency,
created_at, status, type) and settlements.csv (settlement_id,
transaction_id, settled_amount, settled_at, bank_reference).

Detect these gaps:
  - settlement month != transaction month (late settlement)
  - abs(settled_amount - amount) > 0.005 (rounding gap)
  - duplicate transaction_id in transactions (duplicate)
  - refund with type=="refund" where original transaction_id
    not found (orphan refund)
  - transactions with no matching settlement (unmatched)

Output: gaps_report.json and FastAPI endpoint POST /reconcile
that accepts both CSVs and returns gaps as JSON.
Add 4 pytest unit tests using inline DataFrame fixtures.
```

Save this as a plain `.txt` or `.md` file.

---

3. 💻 Claude Code Thread

Full execution thread including wrong turns.

This is the conversation inside Claude Code / Cursor while you were building. They explicitly want the **wrong turns kept in** — errors, retries, failed commands. It shows authentic AI-assisted development.

If you built everything here in Claude.ai (not Claude Code), note that in your submission and attach this chat instead.

---

4. ✅ Test Cases

What you wrote to verify your solution works.

You already have this — it's `tests/test_detectors.py`. Submit it as-is. You can also include the terminal output showing all 15 passing:

```
15 passed in 1.39s
```

Screenshot that output or copy-paste it into a doc.

---

5. 🚀 Working Output

Deployed link + code zip + demo video + 3 sentences.

This has 4 parts:

A) Deployed link Deploy to Render (free, ~5 min):

1. Push your `/reconciler` folder to GitHub
2. Go to render.com → New Web Service → connect repo
3. It reads `render.yaml` automatically → deploy
4. You get a URL like `https://reconcileiq.onrender.com`

B) Code zip Zip the entire `reconciler/` folder:

```
zip -r reconcileiq.zip reconciler/
```

C) Demo video Record a 1–2 min screen recording showing:

- Upload the two CSV files (or click "Load Sample Data")
- Dashboard loads with charts and gap cards
- Filter by gap type
- Download CSV/JSON

- Briefly show the test results

Free tools: Loom (loom.com), QuickTime (Mac), or Xbox Game Bar (Windows).

D) 3 sentences on what it gets wrong in production

Already written for you — copy from the README:

> *The matcher assumes `transaction_id` is a reliable join key, but real payment processors sometimes remap IDs during retries, causing false-positive "unmatched" flags. Rounding detection uses a fixed $0.005 threshold, which breaks for non-USD currencies with different decimal conventions (e.g. JPY has no sub-unit). The duplicate detector compares only the ID field, so a legitimate re-submission with a corrected amount would be silently missed and counted as a clean match.*

Submission Checklist

#	Deliverable	Status	Where it is
1	Brainstorming thread	✅ Ready	This Claude chat — export it
2	Distilled prompt	✅ Ready	Copy from above
3	Claude Code thread	✅ Ready	This chat (or your IDE session)
4	Test cases	✅ Ready	`tests/test_detectors.py` + 15 passed screenshot
5a	Deployed link	⏳ To do	Push to GitHub → deploy on Render
5b	Code zip	⏳ To do	`zip -r reconcileiq.zip reconciler/`
5c	Demo video	⏳ To do	Record with Loom
5d	3 sentences	✅ Ready	Copy from README

The only things left to do are **deploy, zip, and record a 2-minute video.**

3/31/2026, 11:57:29 AM